

Chapter 1

Assignment 1: Syntactic Analysis Pipeline

Legal contracts contain long, complex sentences with nested clauses, coordinating structures, and specialised vocabulary. The first stage of the pipeline decomposes raw contract text into atomic, analysable units and extracts their syntactic structure. Three components are described below: clause splitting, noun-phrase (NP) chunking, and full dependency parsing.

1.1 Clause Splitting

1.1.1 Motivation

A single legal sentence can span several lines and encode multiple independent obligations or conditions. Downstream semantic models (NER, SRL, intent classification) perform significantly better when each input is a single, coherent proposition. The clause splitter reduces complex sentences to their atomic units while preserving the integrity of conditional constructions.

1.1.2 Algorithm

The splitter operates in five sequential stages:

1. **Paragraph segmentation.** The raw input text is partitioned on double newlines, treating each paragraph as an independent processing unit to avoid cross-paragraph sentence boundary errors.
2. **Sentence segmentation.** Each paragraph is processed with spaCy's `en_core_web_sm` model, which detects sentence boundaries more reliably than purely rule-based tokenisers.
3. **Conditional detection.** For each sentence, the algorithm checks whether any token simultaneously satisfies *all three* of the following: (a) its part-of-speech tag is `SCONJ`, (b) its dependency relation is `mark`, and (c) its lower-case form appears in the conditional keyword set (Table 1.1). When this condition is met, the entire sentence is retained as a single indivisible clause. Splitting a conditional sentence would sever the logical dependency between the protasis and apodosis.

4. **Coordinating-conjunction split.** For sentences that do not trigger the conditional guard, the algorithm scans for **CC** tokens whose lower-case form belongs to the coordinate keyword set (Table 1.1). A split is *only* performed when **both** resulting fragments independently contain a subject–verb pair, verified by requiring at least one **nsubj** or **nsubjpass** dependent and one **ROOT** or main-verb token on each side.
5. **Fragment pruning.** Any resulting clause with two tokens or fewer is discarded as a non-informative syntactic fragment.

Table 1.1: Keyword sets governing clause-splitting behaviour.

Set	Keywords
Conditional (SCONJ)	<i>if, unless, provided, whereas, subject, upon, when, whenever, although, though, because, since, after, before, until</i>
Coordinate (CC)	<i>and, but, or, nor, yet, so</i>

1.1.3 Input/Output Examples

Example 1 — Coordinating split. Input:

“The Employer shall pay the Employee a monthly salary of \$5,000 and the Employee shall complete all assigned tasks by the deadline.”

Output — two independent clauses, one per line:

1. *The Employer shall pay the Employee a monthly salary of \$5,000*
2. *the Employee shall complete all assigned tasks by the deadline*

The conjunction *and* carries the **CC** tag. Both resulting fragments contain independent subject–verb pairs (*Employer/shall pay* and *Employee/shall complete*), so a split is performed.

Example 2 — Conditional retained as single clause. Input:

“If the Employee fails to report to work within 3 days without notice, the Employer may terminate this Agreement.”

Output — single clause (no split):

1. *If the Employee fails to report to work within 3 days without notice, the Employer may terminate this Agreement.*

The token *If* carries **SCONJ** POS and **mark** dependency relation, and appears in the conditional keyword set; the sentence is therefore preserved as an atomic unit.

1.2 NP Chunking

1.2.1 Method

Noun-phrase chunking uses spaCy’s built-in `doc.noun_chunks` iterator, which returns contiguous base NP spans derived from the dependency parse tree. These spans are converted to the IOB (**I**nside–**O**utside–**B**eginning) tagging scheme defined as follows:

- **B-NP**: the first (or only) token of a noun phrase span.
- **I-NP**: any subsequent continuation token within the same span.
- **O**: a token that does not belong to any noun phrase.

Output is written to `output/chunks.txt` as tab-separated `token\tIOB-tag` pairs, one pair per line, with a blank line separating successive clauses.

1.2.2 IOB Tagging Example

Table 1.2 shows the IOB annotation produced for the clause “*The contracting parties shall maintain confidentiality.*”.

Table 1.2: IOB tags for “*The contracting parties shall maintain confidentiality.*”

Token	IOB Tag
The	B-NP
contracting	I-NP
parties	I-NP
shall	O
maintain	O
confidentiality	B-NP
.	O

The determiner *The*, the pre-modifying participle *contracting*, and the head noun *parties* together form the subject NP. The modal auxiliary *shall* and main verb *maintain* are outside any NP and are tagged O. The object NP *confidentiality* begins a new span tagged B-NP.

1.3 Dependency Analysis

1.3.1 Parser

Full dependency parsing is performed with spaCy’s `en_core_web_sm` model, which implements a transition-based arc-eager dependency parser trained on the English OntoNotes 5 corpus and annotated with Universal Dependencies (UD) relation labels.

1.3.2 Output Format

Each clause is serialised to a JSON object containing the following top-level fields:

- **clause_id** — zero-based integer index of the clause in the document.
- **text** — the original clause string before tokenisation.
- **tokens** — an array of token objects, each carrying:
 - **id**: position index within the clause (zero-based).
 - **token**: surface form as it appears in the text.
 - **lemma**: morphologically normalised form.
 - **pos**: universal POS tag (e.g. NOUN, VERB).
 - **tag**: fine-grained Penn Treebank tag (e.g. NN, VB, MD).
 - **dep**: Universal Dependencies relation label (e.g. nsubj, dobj, ROOT).
 - **head_id**: position index of the syntactic head token.
 - **head_token**: surface form of the head.

The complete output is written to `output/dependencies.json` as a JSON array of these clause objects.

1.3.3 JSON Output Example

```
1 {
2   "clause_id": 0,
3   "text": "The Employer shall pay the salary.",
4   "tokens": [
5     {"id": 0, "token": "The",      "lemma": "the",
6      "pos": "DET",  "tag": "DT", "dep": "det",
7      "head_id": 1,  "head_token": "Employer"},
8     {"id": 1, "token": "Employer", "lemma": "employer",
9      "pos": "NOUN",  "tag": "NN", "dep": "nsubj",
10     "head_id": 3,   "head_token": "pay"},
11     {"id": 2, "token": "shall",    "lemma": "shall",
12     "pos": "AUX",   "tag": "MD", "dep": "aux",
13     "head_id": 3,   "head_token": "pay"},
14     {"id": 3, "token": "pay",      "lemma": "pay",
15     "pos": "VERB",  "tag": "VB", "dep": "ROOT",
16     "head_id": 3,   "head_token": "pay"},
17     {"id": 4, "token": "the",      "lemma": "the",
18     "pos": "DET",  "tag": "DT", "dep": "det",
19     "head_id": 5,   "head_token": "salary"},
20     {"id": 5, "token": "salary",   "lemma": "salary",
21     "pos": "NOUN",  "tag": "NN", "dep": "dobj",
22     "head_id": 3,   "head_token": "pay"},
23     {"id": 6, "token": ".",        "lemma": ".",
24     "pos": "PUNCT", "tag": ".",  "dep": "punct",
25     "head_id": 3,   "head_token": "pay"}
```

```
26   ]
27 }
```

Listing 1.1: Dependency parse output for the clause “*The Employer shall pay the salary.*”

The ROOT of the clause is *pay* (index 3). *Employer* is its nominal subject (**nsubj**), *shall* is the auxiliary verb (**aux**), and *salary* is the direct object (**dobj**). The determiner *the* (index 0) modifies *Employer* via the **det** relation, while *the* (index 4) modifies *salary* similarly.